

I 1 Đầu ra độ dài thay đổi và sự ra đời của attention (Variable-Length Outputs and the Birth of Attention)

PHỤ THUỘC C/13-kien-truc-backbone/02-tu-chuoi-toi-attention (RNN, LSTM, attention — mục này là *động cơ* của mục đó, nên đọc mục này trước sẽ dễ hơn); B/05-thiet-ke-bai-toan (dạng đầu ra).

TÓM TẮT

Mục này kể lại đúng một sự kiện: **attention** không sinh ra từ mong muốn làm kiến trúc đẹp hơn, mà sinh ra để gỡ một nút thắt rất cụ thể — việc nén cả một ảnh vào một vector duy nhất trước khi sinh câu mô tả. Đọc xong bạn hiểu *vì sao* self-attention của Transformer là hiển nhiên chứ không bí ẩn, và bạn nhận ra được cùng một nút thắt ở những nơi khác, kể cả trong pipeline SLAM của chính mình. Nếu chỉ nhớ một điều: ở đâu một hệ thống nén thông tin có cấu trúc thành một vector cố định *trước khi* biết sẽ cần phần nào, ở đó có một nút thắt. Attention là cách chuẩn để gỡ nó, bằng cách hoãn phép nén tới lúc biết đang cần gì.

KIẾN THỨC CẦN CÓ TRƯỚC

- **token** — một đơn vị rời rạc của chuỗi đầu ra — thường là một từ hoặc một mảnh từ. Mô hình sinh câu bằng cách dự đoán token tiếp theo, lặp lại tới khi gặp token kết thúc.
- **RNN và LSTM** — mạng xử lý chuỗi bằng cách mang theo một *trạng thái ẩn* (một vector) từ bước này sang bước sau. LSTM là biến thể có cổng, giúp trạng thái giữ được thông tin qua nhiều bước hơn.
- **encoder-decoder** — encoder biến đầu vào thành biểu diễn số; decoder sinh đầu ra từ biểu diễn đó. Ở đây encoder là CNN nhìn ảnh, decoder là mạng sinh câu.
- **feature map (lưới đặc trưng)** — đầu ra không gian của CNN: một lưới, ví dụ 14×14 , mỗi ô là một vector mô tả một vùng ảnh. Giữ nguyên lưới nghĩa là giữ nguyên thông tin “ở đâu”.
- **global average pooling** — phép lấy trung bình mọi ô của lưới đặc trưng để ra một vector duy nhất. Rất tiện cho phân loại, nhưng nó xóa sạch thông tin vị trí.
- **attention** — cơ chế cho một bên (bên cần thông tin) tính một bộ trọng số trên các phần của bên kia (bên có thông tin), rồi lấy tổ hợp có trọng số. Trọng số phụ thuộc vào việc đang cần gì, nên phép nén được “hoãn” tới lúc có truy vấn.
- **self-attention** — trường hợp hai bên là cùng một tập: mỗi phần tử tự tính trọng số trên mọi phần tử khác của chính chuỗi đó. Đây là khối cơ bản của Transformer.

Đặt vấn đề

Toàn bộ chương 14 giả định một điều mà ta không để ý: đầu ra tuy có thể là tập hợp nhưng *mỗi phần tử* của nó vẫn có hình dạng cố định — một khung là bốn số, một mask là một lưới nhị phân. **Sinh mô tả ảnh** (*image captioning*) phá vỡ cả điều đó: đầu ra là một câu, dài ngắn tùy ảnh, và mỗi từ phụ thuộc vào các từ trước. Đây là lần đầu thị giác máy tính phải sinh ra một chuỗi, và nó buộc phải mượn bộ máy của xử lý ngôn ngữ.

Giải pháp đầu tiên nghe rất hợp lý. Cho ảnh qua CNN, lấy vector đặc trưng ở tầng áp chót, dùng nó làm trạng thái khởi tạo cho một **mạng hồi quy** (*recurrent neural network, RNN*) hoặc **LSTM** [1], rồi để mạng đó sinh từng **token** một. Kiến trúc này chạy được và cho ra những câu nghe khá trôi chảy. Nhưng nó chứa một giả định chết người: rằng *một* vector cố định — thường 512 hay 2048 số — đủ để trả lời mọi câu hỏi mà mọi từ trong câu sẽ đặt ra. Khi mạng đang sinh từ “con chó”, nó cần biết chi tiết ở góc dưới bên trái; khi sinh từ “bãi cỏ”, nó cần chi tiết ở nền. Vector kia phải chứa sẵn cả hai, và cả những chi tiết cho mọi câu khác mà nó không biết trước.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn phát hiện được nút thắt kiểu “nén trước, dùng sau” trong một kiến trúc bất kỳ, kể cả kiến trúc không dính gì tới ngôn ngữ. Bạn cũng biết attention là công cụ chuẩn để gỡ nó, và biết cái giá phải trả khi gỡ.

TRỰC GIÁC

Hãy tưởng tượng bạn phải kể lại một bộ phim cho người khác, nhưng luật chơi là: xem xong, bạn chỉ được ghi *một* câu tóm tắt vào giấy, rồi trả lại bằng phim, và từ đó chỉ được nhìn tờ giấy để kể suốt hai mươi phút. Bạn sẽ kể trôi chảy, nhưng mọi chi tiết đều là bịa hợp lý. Attention là việc bỏ luật đó đi: mỗi khi kể đến một cảnh, bạn được phép tua lại băng và nhìn đúng cảnh ấy. Không có gì huyền bí — toàn bộ ý tưởng chỉ là *hoãn việc nén lại cho tới khi biết mình đang cần gì*.

A. Từ một vector tới một cái nhìn có chọn lọc

I Mạch 1 — đầu ra độ dài thay đổi

- 1 Vấn đề** đầu ra là một câu, không phải một nhãn.

Giải pháp gán một bộ giải mã chuỗi lên trên bộ trích đặc trưng: CNN cho vector, RNN/LSTM sinh từng token, huấn luyện bằng **entropy chéo** (*cross-entropy*) trên từng vị trí token.

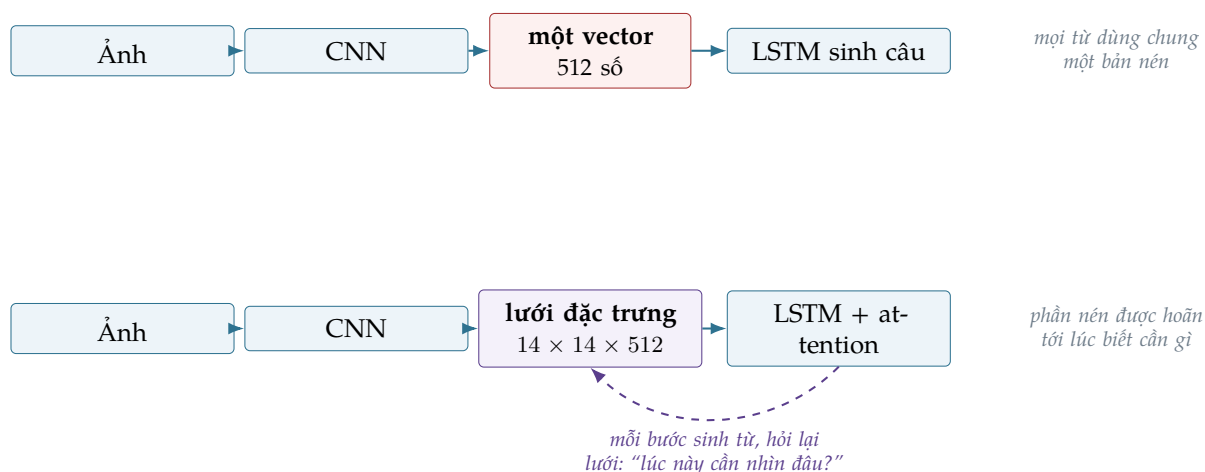
Giả định đặc trưng toàn cục của ảnh chứa đủ thông tin cho toàn bộ câu.

Hệ quả lần đầu tiên một hệ thị giác sinh được đầu ra có độ dài thay đổi.
- 2 Vấn đề** một vector duy nhất là nghẽn cổ chai thông tin. Mọi chi tiết không gian bị nén trước khi ta biết sẽ cần chi tiết nào; và nén *có mất mát* thì không đảo ngược được.

Giải pháp “Show, Attend and Tell” giữ nguyên lưới đặc trưng không gian [2]. Ở mỗi bước sinh từ, bộ giải mã tính một phân bố trọng số trên các ô lưới. Tổ hợp có trọng số của các ô đó là đầu vào cho bước ấy.

Hệ quả đây là nơi attention xuất hiện lần đầu trong thị giác, và nó ra đời để gỡ một nút thắt cụ thể chứ không phải vì đẹp. Ý tưởng gốc đến từ dịch máy, nơi cùng một nghẽn cổ chai xuất hiện ở dạng vector câu nguồn [3].
- 3 Vấn đề** nếu attention hữu ích cho quan hệ giữa *câu và ảnh*, tại sao không dùng cho quan hệ giữa *các phần của cùng một ảnh*?

Giải pháp đó chính là bước đi tới self-attention và **transformer** [4], và đó là lý do nên đọc mục này *trước* C/13-kien-truc-backbone/02-tu-chuoi-toi-attention: khi đã thấy nút thắt, self-attention trở thành hệ quả hiển nhiên thay vì một phát minh từ trên trời rơi xuống.



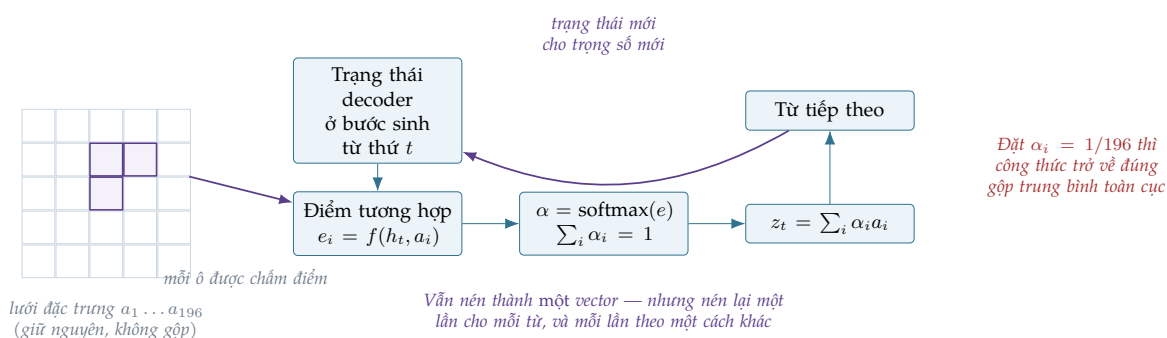
Hình 1. Cùng một pipeline, khác nhau đúng một chỗ. Nhánh trên nén lưới đặc trưng thành một vector trước khi biết câu sẽ nói gì; nhánh dưới giữ nguyên lưới và để bộ giải mã truy vấn lại nó ở mỗi bước. Mỗi tên dứt là toàn bộ nội dung của attention — một vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

B. Nút thắt lớn tới mức nào — một phép tính

Hình 1 sẽ thuyết phục hơn nhiều khi kèm con số. Lưới đặc trưng của một CNN trước tầng gộp toàn cục có kích thước điển hình $14 \times 14 \times 512$, tức $196 \times 512 = 100\,352$ số. Sau **gộp trung bình toàn cục** (*global average pooling*) chỉ còn 512 số. Tỉ số nén là $196:1$, và điều quan trọng không phải tỉ số mà là *cái gì bị mất*: phép gộp lấy trung bình theo không gian, nên nó xoá sạch thông tin “đặc trưng này nằm ở đâu”. Một câu mô tả cần đúng thông tin đó để nói “con chó bên trái người đàn ông”.

↔ hệ quả của C/13/02 — self-attention là bước tiếp theo hiển nhiên khi đem chính cơ chế này áp cho quan hệ giữa các phần bên trong một đầu vào, thay vì giữa hai đầu vào khác nhau.

Nói gọn thì attention thay phép trung bình đều bằng phép trung bình có trọng số, với trọng số phụ thuộc vào việc bộ giải mã đang cần gì: gọi a_i là vector đặc trưng của ô thứ i , và α_i là trọng số attention tại bước sinh từ hiện tại. Đầu vào của bước đó là $z = \sum_i \alpha_i a_i$, với $\sum_i \alpha_i = 1$. Công thức này nói một điều rất đơn giản: *vẫn nén thành một vector, nhưng nén lại một lần cho mỗi từ, và mỗi lần nén theo một cách khác nhau*. Trường hợp $\alpha_i = 1/196$ với mọi i chính là gộp trung bình toàn cục — mô hình cũ là một trường hợp riêng của mô hình mới.



Hình 2. Cơ chế attention viết ra thành luồng tính. Điều đáng chú ý không phải công thức mà là vòng phản hồi: trạng thái của bộ giải mã quyết định trọng số, trọng số quyết định vector đưa vào bước sau, và bước sau lại đổi trạng thái. Mô hình cũ là trường hợp riêng khi trọng số bị ép bằng nhau, nên đây là một phép tổng quát hoá chứ không phải một kiến trúc cạnh tranh.

Hình 2 cũng cho thấy chi phí: phần chấm điểm phải chạy lại cho *mỗi* từ trên *mọi* ô lưới, nên tổng chi phí tỉ lệ với tích số ô nhân số bước sinh — mầm mống của vấn đề bậc hai ở mục sau.

Bảng 1. Ba cách đưa thông tin ảnh vào bộ giải mã chuỗi.

Cách	Cơ chế	Giá phải trả	Hỏng khi nào
Một vector toàn cục	Gộp trung bình rồi khởi tạo trạng thái	Rẻ nhất, một lần tính	Câu cần chi tiết cục bộ hoặc quan hệ không gian; mô hình bị chi tiết hợp lý
Soft attention	Trung bình có trọng số, khả vi	Chi phí tỉ lệ số ô $\times$ số từ	Ảnh nhiều vật giống nhau: trọng số dàn đều, không chọn được
Hard attention	Lấy mẫu rồi rạc một ô	Rẻ khi chạy, nhưng không khả vi	Phải huấn luyện bằng ước lượng gradient có phương sai cao; kém ổn định

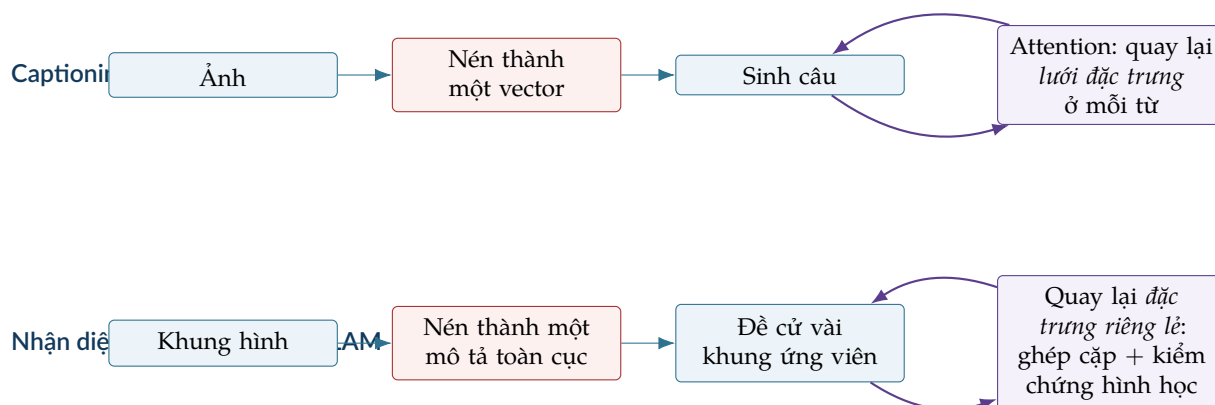
↔ cùng nguyên lý với C/14/02 — mỗi truy vấn của DETR “hỏi” feature map xem vật của nó ở đâu — đúng vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

C. Cùng một nút thắt trong pipeline SLAM

Chỗ nổi này đáng dừng lại, vì nó cho thấy nút thắt trên không phải chuyện riêng của ngôn ngữ. Một hệ *SLAM* nhận diện nơi đã đi qua bằng cách nén cả khung hình thành một mô tả toàn cục — túi từ trực quan hoặc một vector nhúng — rồi so mô tả đó với cơ sở dữ liệu [5], [6]. Bước nén ấy có *đúng* tính chất của nghẽn cổ chai: nó xảy ra trước khi biết sẽ cần kiểm chứng điều gì, nên nó vứt bỏ hình học.

Và giải pháp mà cộng đồng SLAM dùng cũng có đúng cấu trúc của attention: sau khi mô tả toàn cục đề cử vài ứng viên, hệ thống *quay lại* các đặc trưng riêng lẻ để ghép cặp và kiểm chứng bằng ràng buộc hình học. Nói cách khác, mô tả toàn cục chỉ dùng để *chọn nơi cần nhìn kỹ*, chứ không dùng để kết luận — y hệt vai trò của phân bố α trong công thức trên. Nếu bạn thấy self-attention khó hình dung, hãy nghĩ nó như bước kiểm chứng đặc trưng đó, chỉ khác là trọng số được học thay vì được tính bằng RANSAC.

↔ cùng nguyên lý với C/16 — nhận diện nơi đã đi qua nén khung hình thành một mô tả toàn cục rồi phải quay lại đặc trưng để kiểm chứng hình học — cùng cấu trúc “đề cử rồi nhìn kỹ” của attention.



Cùng một cấu trúc: bản nén toàn cục chỉ dùng để chọn nơi cần nhìn kỹ, không dùng để kết luận. Khác biệt duy nhất là trọng số quay lại được học (attention) hay được tính bằng ràng buộc hình học (RANSAC).

Hình 3. Nghẽn cổ chai “nén trước, dùng sau” không phải chuyện riêng của ngôn ngữ. Hệ SLAM gặp đúng nó khi nén một khung hình thành một mô tả toàn cục để tra cứu, và giải đúng theo cách của attention: dùng bản nén để đề cử, rồi quay về nguồn thông tin đầy đủ để kiểm chứng. Nhận ra được cấu trúc này thì self-attention thôi trông giống một phát minh xa lạ.

Hình 3 là lý do mục này nên đọc trước phần Transformer: khi mẫu thiết kế đã quen, phần công thức chỉ còn là chi tiết.

D. Giới hạn và trường hợp hỏng

- **Thước đo của captioning rất yếu.** Các độ đo dựa trên trùng khớp n -gram thường cho câu “an toàn”: đúng ngữ pháp và giống câu tham chiếu. Vì thế một mô hình sinh mô tả chung chung có thể ghi điểm cao hơn một mô hình mô tả đúng nhưng diễn đạt khác. *Hệ quả thực tế:* cải tiến trên bảng xếp hạng không bảo đảm cải tiến khi người dùng thật đọc.
- **Mô hình bịa chi tiết.** Vì bộ giải mã được huấn luyện để sinh câu *hợp lý*, nó lấp chỗ trống bằng thứ thường đúng — “người đàn ông cầm ván lướt sóng” cho một ảnh bãi biển không có ván. Nút thắt càng chặt thì hiện tượng này càng nặng, nên đây là triệu chứng chẩn đoán được.
- **Bản đồ attention không phải lời giải thích.** Rất dễ nhìn α rồi tin rằng mô hình “đang nhìn vào đó vì lý do đó”. Các kiểm tra tính đúng đắn cho thấy nhiều loại bản đồ nổi bật vẫn trông hợp lý ngay cả khi trọng số mô hình bị làm hỏng [7], nên trọng số attention là *manh mối chẩn đoán*, không phải bằng chứng.
- **Chi phí tăng theo tích số ô $\times$ số bước.** Với ảnh phân giải cao hoặc câu dài, phần attention có thể vượt chi phí của chính CNN. Đây là mầm mống của vấn đề bậc hai ở mục sau, khi số ô còn nhân thêm với số khung hình.

CẦN NHỚ

Nghẽn cổ chai thông tin xuất hiện ở *mọi* chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định trước khi biết sẽ cần phần nào. Attention không phải một loại tầng mạng, nó là một mẫu thiết kế: *giữ nguyên nguồn, và hoãn việc nén tới lúc có truy vấn*. Nhận ra mẫu này rồi thì Transformer, cross-attention trong DETR, và cả bước kiểm chứng hình học trong SLAM đều là cùng một thứ. [T2]

E. Kết nối

C/13-kien-truc-backbone/02-tu-chuoi-toi-attention là nơi hình thức hoá đầy đủ; mục này là động cơ của nó, nên thứ tự đọc tự nhiên là mục này trước.

C/14-bai-toan-cv-loi/02-detection-mot-giai-doan dùng lại chính cơ chế này dưới dạng truy vấn của DETR: mỗi truy vấn “hỏi” feature map xem vật của nó ở đâu. C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video là bước kế tiếp trực tiếp: khi nguồn không chỉ là một lưới không gian mà là một khối không-thời gian, chi phí attention nhân lên và trở thành ràng buộc thiết kế chính.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nén ảnh thành một vector *trước* khi sinh câu lại là một quyết định tồi, trong khi nén ảnh thành một vector trước khi phân loại thì hoàn toàn ổn?
2. Chứng minh bằng một câu rằng gộp trung bình toàn cục là trường hợp riêng của soft attention. Điều đó nói gì về quan hệ giữa hai kiến trúc?
3. Hard attention rẻ hơn khi chạy nhưng ít được dùng. Vì sao?
4. Vì sao không nên đọc bản đồ attention như một lời giải thích về nguyên nhân?
5. Trong một hệ thống bạn đã làm, nghẽn cổ chai kiểu “nén trước, dùng sau” nằm ở đâu, và hệ thống đó bù lại bằng bước “quay lại nguồn” nào?

Nguồn

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [5] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [6] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [7] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.